NOM DEL PROJECTE / ACTIVITAT

GameVault v1: completa el gestor del catàleg — PROVA PRÀCTICA DEL RA1

CODIFICACIÓ

PR1_AEA1_MP0486

RESULTATS D'APRENENTATGE

El resultat d'aprenentatge que es treballa en aquesta activitat és:

RA1. Desenvolupa aplicacions que gestionen informació emmagatzemada en fitxers identificant-ne el camp d'aplicació i utilitzant classes específiques.

Es treballen **tots els criteris d'avaluació del RA1**. La taula indica on es demostra cadascun:

CA	Criteri d'avaluació	On es demostra
1.1	Utilitza classes per a la gestió de fitxers i directoris.	PR1-02, PR1-03, PR1-04
1.2	Valora els avantatges i els inconvenients de les diferents formes d'accés.	PR1-12, PR1-13 i pregunta 1 del README
1.3	Utilitza classes per recuperar informació emmagatzemada en fitxers.	PR1-06, PR1-09 i PR1-11 (carrega)
1.4	Utilitza classes per emmagatzemar informació en fitxers.	PR1-05, PR1-07, PR1-08 i PR1-11 (desa)
1.5	Utilitza classes per fer conversions entre diferents formats de fitxers.	PR1-08, PR1-10 i pregunta 2 del README
1.6	Preveu i gestiona les excepcions.	PR1-01, PR1-06, PR1-10 i pregunta 3 del README
1.7	Prova i documenta les aplicacions desenvolupades.	PR1-14, PR1-15, Javadoc i historial de commits

ENUNCIAT

GameVault ja té totes les peces que hem anat construint a classe: les carpetes del magatzem, el catàleg en CSV, el JSON, la cau binària i l'índex d'hores. Ara cal muntar-les en una sola aplicació que funcioni de principi a fi.

L'equip ja ha deixat fet l'esquelet. El projecte **compila**, té tots els paquets i les classes, el programa principal `GameVaultApp` i **25 proves automàtiques**. Però hi falten els trossos de codi que fan la feina de veritat. Estan marcats al codi amb `TODO PR1-01` fins a `TODO PR1-13`.

La teva feina és **completar aquests forats**, escriure **dues proves teves** i respondre **tres preguntes** al README. Cada forat correspon a un exemple que ja has vist a classe (sessions 1 a 6): sempre tens un model per consultar al repositori d'exemples del mòdul.

Aquesta és la prova pràctica del RA1. Val el **80 %** de la nota del RA1.



Figura 1. Les classes del projecte i el forat que has de completar a cadascuna.

Desenvolupament de la tasca

Per desenvolupar aquesta activitat has de dur a terme els passos següents.

Pas 1 · Prepara el projecte

1. Baixa **gamevault-pr1.zip** de l'aula del Classroom i descomprimeix-lo.
2. Obre la carpeta amb **IntelliJ IDEA** (*File → Open*). És un projecte Maven amb Java 21: IntelliJ baixa sol les llibreries que fa servir (OpenCSV, Jackson i JUnit).
3. Crea un repositori a **GitHub**, puja-hi el projecte tal com el reps i fes el primer commit: `Projecte de partida de la PR1`.
4. Executa totes les proves: clic dret a `src/test/java` → *Run 'All Tests'*. Han de sortir **27 proves** i gairebé totes en vermell. És normal: encara no has escrit res.
5. Per veure la llista de forats: *View → Tool Windows → TODO*, o cerca `PR1-` a tot el projecte (`Ctrl+Maj+F`).

Pas 2 · Entén com funciona un forat

Cada forat té un comentari que comença per `TODO PR1-xx`. El comentari diu què has de fer i de quin exemple de classe surt. A dins del mètode hi ha una línia provisional que has de substituir pel teu codi:

IndexHores.java · tal com el reps

```
/*
 * TODO PR1-12 · Llegir només les hores d'un registre (CA 1.2 i 1.3)
 * Obre el fitxer amb RandomAccessFile en mode lectura, fes seek
 * directament al camp de les hores del registre i llegeix el double. ...
 */
public double llegeixHores(int posicio) throws IOException {
    // Escriu aquí el teu codi (PR1-12)
    throw new UnsupportedOperationException("PR1-12: encara no està fet");
}
```

Fragment	Què fa
`/* TODO PR1-12 … */`	L'enunciat del forat, dins del codi. Quan l'hagis resolt, substitueix aquest comentari per un Javadoc teu (vegeu el pas 4).
`public double llegeixHores(int posicio)`	La signatura del mètode. **No la canviïs**: les proves i GameVaultApp el criden exactament així.
`throws IOException`	Avisa que el mètode pot llançar errors d'entrada i sortida. Els tracta qui el crida. Tampoc no l'has de treure.
`throw new UnsupportedOperationException(…)`	Línia provisional: fa que el projecte compili encara que el mètode estigui buit. Si s'executa, la prova falla amb el missatge «PR1-12: encara no està fet». L'has d'esborrar quan escriguis la teva solució.

Les proves ja estan escrites i et diuen si el forat funciona. Aquesta és una de les que comproven el PR1-12:

IndexHoresTest.java · no es modifica

```
@Test
@DisplayName("PR1-12 · llegeix les hores de qualsevol registre")
void llegeixHores() throws Exception {
    assertEquals(42.5, index.llegeixHores(0));
    assertEquals(12.5, index.llegeixHores(2));
    assertEquals(120.0, index.llegeixHores(4));
}
```

Fragment	Què fa
`@Test`	Marca el mètode com una prova de JUnit 5. IntelliJ l'executa i el pinta de verd o de vermell.
`@DisplayName("PR1-12 · …")`	El nom que veus a la finestra de proves. Sempre comença pel codi del forat que comprova.
`assertEquals(42.5, index.llegeixHores(0))`	Espera que el registre 0 (Hollow Knight) tingui 42,5 hores. Si el teu mètode retorna una altra cosa, la prova surt en vermell i et mostra el valor esperat i el que ha obtingut.

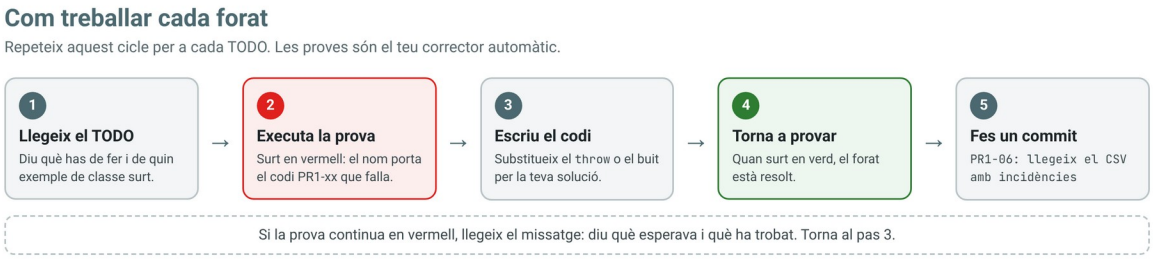


Figura 2. El cicle de treball de cada forat.

Pas 3 · Completa els forats, part per part

Fes les parts en aquest ordre. Algunes proves en necessiten de prèvies: per exemple, el Convertidor fa servir els DAO de CSV i de JSON, i la lectura del CSV fa servir la validació del Joc.

Part A · El magatzem de fitxers (CA 1.1)

Classe `fs/Magatzem`. Model: sessió 2 (`Ex3CreaCarpetes`, `Ex4CopiaMouEsborra` i `Ex5Recorre`).

- **PR1-02** `preparaEstructura()`: crea les carpetes `import`, `export` i `backup`. Si ja existeixen, no pot fallar.
- **PR1-03** `copiaSeguretat(Path)`: copia el fitxer a `backup` amb el nom `nom_marca.extensió` (per exemple `catalog_20261013-151002.csv`) i retorna la ruta de la còpia. L'original no es toca.
- **PR1-04** `midaTotal()`: suma les mides de tots els fitxers del magatzem, també els de les subcarpetes.

Codi que ja tens fet i que et serà útil:

Magatzem.java · ja fet

```
public Path copies() {
    return base.resolve("backup");
}

public static String marcaDeTemps() {
    return LocalDateTime.now().format(MARCA);    // MARCA = "yyyyMMdd-HHmmss"
}
```

Fragment	Què fa
`base.resolve("backup")`	Construeix la ruta `dades/backup` a partir de la carpeta base. **No toca el disc**: només crea l'objecte Path. Per crear la carpeta cal un mètode de Files.
`LocalDateTime.now().format(MARCA)`	Data i hora actuals com a text, per exemple `20261013-151002`. És la marca que va al nom de la còpia.

Proves: `MagatzemTest` (3).

Part B · El model i el registre d'operacions (CA 1.6 i 1.4)

- **PR1-01** `Joc`: valida al constructor compacte que el títol no sigui buit, que les hores no siguin negatives i que la nota vagi de 0 a 10. Si no es compleix, llança `IllegalArgumentException` amb un missatge que contingui «títol», «hores» o «nota». Fes-lo primer: la lectura del CSV en depèn.
- **PR1-05** `RegistreOperacions.registra(String)`: afegeix una línia al final de `operacions.log` amb aquest format, sense esborrar les anteriors. Model: sessió 5, `Ex1FluxosDeText`.

Una línia del registre

```
2026-10-13 15:10:02 | còpia de seguretat catalog_20261013-151002.csv
```

Fragment	Què fa
`DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss")`	La constant `FORMAT` de la classe. Converteix una data i hora en el text del principi de la línia.
`LocalDateTime.now().format(FORMAT)`	Dona el text `2026-10-13 15:10:02` amb el moment actual.

Proves: `JocTest` (4) i `RegistreOperacionsTest` (2).

Part C · El catàleg en CSV (CA 1.3, 1.4 i 1.6)

Classe `io/CsvJocDao`. Model: sessió 5 (`Ex3LlegeixCsvBrut` i `Ex4EscriuCsv`).

- **PR1-06** dins de `llegeix(Path)`: converteix cada línia en un `Joc`. Si la línia és dolenta, afegeix una incidència «línia N: motiu» i continua amb la següent.
- **PR1-07** `escriu(Path, List<Joc>)`: escriu la capçalera i una fila per joc amb `CSVWriter`.

El fitxer `dades/catalog_brut.csv` és el de l'AA2. Això és el que ha de passar amb cada línia:

Línia	Contingut	Resultat esperat
2	`1;Hollow Knight;…;42.5;9.5`	Entra al catàleg
3	`2;Celeste;…;15.0;12.0`	Descartada: el Joc rebutja la nota (el motiu conté «nota»)
4	`3;Gris;PC`	Descartada: incompleta (només 3 camps)
5	`4;Hades;…;25-09-2020;…`	Descartada: data mal formada
6	`5;"Baldur's Gate 3; Deluxe";…`	Entra: les cometes protegeixen el «;» del títol
7	`6;Metroid Dread;…;-5.0;9.0`	Descartada: el Joc rebutja les hores (el motiu conté «hores»)

El bucle de lectura ja el tens fet. Només has d'escriure el que va a dins:

CsvJocDao.llegeix · ja fet

```
CSVParser parser = new CSVParserBuilder().withSeparator(SEPARADOR).withQuoteChar('"').build();

try (Reader r = Files.newBufferedReader(p, StandardCharsets.UTF_8);
     CSVReader csv = new CSVReaderBuilder(r).withCSVParser(parser).withSkipLines(1).build()) {

    String[] c;
    int linia = 1;                      // la línia 1 és la capçalera
    while ((c = csv.readNext()) ≠ null) {
        linia++;
        // Escriu aquí el teu codi (PR1-06)
    }
}
```

Fragment	Què fa
`withSeparator(SEPARADOR).withQuoteChar('"')`	El parser sap que els camps se separen amb «;» i que el text entre cometes és un sol camp encara que porti un «;».
`Files.newBufferedReader(p, StandardCharsets.UTF_8)`	Obre el fitxer per llegir text amb la codificació explícita, perquè els accents surtin igual a tots els ordinadors.
`withSkipLines(1)`	Salta la capçalera: el primer `readNext()` ja retorna la línia 2.
`while ((c = csv.readNext()) ≠ null)`	Llegeix una línia cada volta i la deixa a `c` com un array de camps. Quan s'acaba el fitxer retorna null i el bucle para.
`linia++`	Porta el número de línia del fitxer, que necessites per a les incidències.

Per a PR1-07 tens fet el mètode `aCamps(Joc)`, que converteix un joc en els 7 textos de la fila.

Proves: `CsvJocDaoTest` (4).

Part D · JSON i conversió de formats (CA 1.4, 1.5 i 1.6)

Classes `io/JsonJocDao` i `servei/Convertidor`. Model: sessió 6 (`Ex4JsonAmbJackson`).

- **PR1-08** configura l'`ObjectMapper` (dates com «2017-02-24» i tolerància a camps de més) i escriu la llista en JSON indentat.
- **PR1-09** llegeix el JSON i retorna una `List<Joc>` (no una llista de mapes).
- **PR1-10** al Convertidor: escriu `desa()` seguint el model de `carrega()`, que ja està fet, i després `converteix()`, que carrega l'origen, el desa al destí i retorna quants jocs ha convertit.

Convertidor.carrega · ja fet, et serveix de model

```
public List<Joc> carrega(Path origen) throws CatalegException {
    try {
        return switch (extensio(origen)) {
            case "csv" → csv.llegeix(origen).jocs();
            case "json" → json.llegeix(origen);
            default → throw new CatalegException("Format no suportat: " + origen.getFileName());
        };
    } catch (IOException e) {
        throw new CatalegException("No s'ha pogut llegir " + origen.getFileName()
                + ": " + e.getMessage(), e);
    }
}
```

Fragment	Què fa
`switch (extensio(origen))`	Tria què fer segons l'extensió del fitxer, en minúscules: "csv", "json" o una altra.
`case "csv" → csv.llegeix(origen).jocs()`	Un CSV es llegeix amb el DAO de CSV. Només interessen els jocs vàlids, no les incidències.
`default → throw new CatalegException(…)`	Qualsevol altra extensió és un error previsible: es llança l'excepció pròpia amb un missatge que l'usuari entén.
`catch (IOException e)`	Si el DAO falla (fitxer inexistent, JSON trencat), l'error tècnic es converteix en una CatalegException.
`new CatalegException(missatge, e)`	El segon paràmetre guarda l'excepció original com a **causa**: no es perd la informació tècnica.

Proves: `JsonJocDaoTest` (3) i `ConvertidorTest` (3). Les del Convertidor necessiten els forats 06 a 09.

Part E · La cau binària (CA 1.3 i 1.4)

Classe `io/CauCataleg`. Model: sessió 6 (`Ex1Serialitza`).

- **PR1-11** `desa()` escriu tota la llista amb `ObjectOutputStream` i `carrega()` la recupera amb `ObjectInputStream`. Si `readObject` llança `ClassNotFoundException`, embolica-la en una `IOException` amb la causa.
- Segona part del PR1-11: perquè es pugui serialitzar, el record `Joc` ha d'implementar `Serializable`. Si te n'oblides, la prova falla amb `NotSerializableException`.

Una de les proves comprova que el fitxer comença pels bytes `AC ED`. Tots els fitxers de serialització Java comencen així: és la seva «signatura». Si escrius el fitxer d'una altra manera, la prova ho detecta.

Proves: `CauCatalegTest` (3).

Part F · L'índex d'hores amb accés aleatori (CA 1.2)

Classe `io/IndexHores`. Model: sessió 5 (`Ex5AccesAleatori`). Cada registre fa sempre 92 bytes:

Camp	Tipus	Bytes	Com s'escriu
id	int	4	writeInt
títol	40 caràcters	80	writeChars (2 bytes per caràcter)
hores	double	8	writeDouble
Total		**92**	constant MIDA_REGISTRE

- **PR1-12** `llegeixHores(int)`: fes `seek` directament al camp de les hores del registre i llegeix el `double`. Pensa quants bytes hi ha dins del registre **abans** d'aquest camp.
- **PR1-13** `sumaHores(int, double)`: llegeix les hores, suma-hi l'extra i escriu el valor nou **al mateix lloc**. Cap altre byte del fitxer pot canviar.

IndexHores.escriu · ja fet

```
public void escriu(int posicio, Joc j) throws IOException {
    try (RandomAccessFile raf = new RandomAccessFile(fitxer.toFile(), "rw")) {
        raf.seek((long) posicio * MIDA_REGISTRE);
        raf.writeInt(j.id());
        String fix = String.format("%-" + LONG_TITOL + "s", j.titol()).substring(0, LONG_TITOL);
        raf.writeChars(fix);
        raf.writeDouble(j.horesJugades());
    }
}
```

Fragment	Què fa
`new RandomAccessFile(…, "rw")`	Obre el fitxer per llegir i escriure en qualsevol posició. Amb "r" només es pot llegir.
`raf.seek((long) posicio * MIDA_REGISTRE)`	Mou el punter al primer byte del registre. El `(long)` evita que la multiplicació es desbordi amb fitxers grans.
`String.format("%-40s", …).substring(0, 40)`	Omple el títol amb espais fins a 40 caràcters, o el talla. Així tots els registres pesen el mateix i es poden localitzar amb una multiplicació.
`writeInt · writeChars · writeDouble`	Escriuen els tres camps un darrere l'altre. Cada escriptura avança el punter tants bytes com ha escrit.

Proves: `IndexHoresTest` (3). La darrera comprova byte a byte que només han canviat els 8 bytes del camp.

Pas 4 · Proves i documentació (CA 1.7)

- **PR1-14** Escriu **dues proves teves** a `ProvesPropiesTest`, que no repeteixin les que ja hi ha. Canvia el `@DisplayName` perquè digui què comproves i treu el `fail()`. Idees: un títol de més de 40 caràcters a l'índex, la còpia d'un fitxer sense extensió, un CSV amb

una línia en blanc, carregar un .json que no existeix amb el Convertidor, sumar hores a dos registres seguits.

- **PR1-15** Respon al `README.md` les tres preguntes: accés seqüencial i aleatori (CA 1.2), quin format triar per a cada ús (CA 1.5) i què passa amb una línia del CSV amb la data mal escrita (CA 1.6). Mínim 4 línies cadascuna, amb les teves paraules.
- **Javadoc.** Substitueix el comentari `/* TODO … */` de cada mètode que completis per un Javadoc teu en català (`/** … */`) amb `@param`, `@return` i `@throws`.
- **Ús d'IA.** Completa la declaració del final del README: si has fet servir alguna eina d'IA, per a què i en quins forats.

Pas 5 · Comprova-ho tot i lliura

1. Executa totes les proves: les **27** han de sortir en verd.
2. Executa `GameVaultApp` (▶️ al costat del `main`) i compara la sortida amb la de l'annex 1.
3. Fes l'últim commit i puja'l a GitHub.
4. Comprimeix el projecte **sense la carpeta target** i lliura'l al Classroom amb l'enllaç del repositori.

Perquè l'activitat es consideri completa has de:

- Tenir els **13 forats** completats i el projecte compilant sense errors.
- Tenir les **25 proves donades en verd**, sense haver-les modificat.
- Tenir **2 proves pròpies** significatives en verd a `ProvesPropiesTest`.
- Que `GameVaultApp` doni la sortida de l'annex 1 (només poden canviar la marca de temps i alguna mida).
- Tenir les **3 respostes** i la declaració d'ús d'IA al README.
- Tenir **Javadoc en català** als mètodes que has completat.
- No tenir cap `catch` buit ni cap `printStackTrace()`, i fer servir **try-with-resources** a tots els fluxos.
- Tenir un mínim de **5 commits** repartits entre les sessions.

RECURSOS

- Projecte de partida **gamevault-pr1.zip** (aula del Classroom).
- Repositori d'exemples del mòdul: **github.com/ilopez000/0486-acces-a-dades** (paquets `sessio2`, `sessio5` i `sessio6`).
- Recursos de sessió a l'aula: *Path i Files* (sessió 2), *Fluxos, CSV amb OpenCSV i accés aleatori* (sessió 5) i *Serialització d'objectes i JSON amb Jackson* (sessió 6).
- Apunts **0486_AEA1_Apunts_GestioDeFitxersAmbJava**.
- Escape room de repàs *GameVault · Save corrupted*: **0486-escape-room-gamevault.pages.dev**.

- Java 21, IntelliJ IDEA, Maven, Git i GitHub. Documentació oficial de l'API de Java, d'OpenCSV i de Jackson. **Consulta permesa.**

CONEIXEMENTS PREVIS

- `record` i validació al constructor compacte (sessió 1).
- `java.nio.file`: `Path`, `Files` i recorregut de carpetes (sessions 2 i 3).
- Fluxos de caràcters i de bytes, i `try-with-resources` (sessió 4).
- CSV amb OpenCSV i fitxers d'accés aleatori amb `RandomAccessFile` (sessió 5).
- Serialització d'objectes i JSON amb Jackson (sessió 6).
- Excepcions comprovades i no comprovades. Executar proves de JUnit des d'IntelliJ.

INSTRUMENTS D'AVALUACIÓ

- **Prova pràctica avaluable**, corregida amb la rúbrica de l'annex 3 (també a l'aula en Excel).
- **Ponderació: 80 % del RA1.** El 20 % restant és el PE1 (test del RA1).
- Nota del RA1 = PR1 × 0,8 + PE1 × 0,2. El test pondera a partir de 4 i cal un 5 al RA1 per fer mitjana amb la resta de RA del mòdul.
- Les proves automàtiques ajuden a corregir, però no posen la nota soles: el docent revisa el codi, els commits i el README, i pot demanar que expliquis qualsevol forat.
- **Metodologia:** EM_03 Treballs i casos pràctics (activa) · EM_04 Ús d'eines TIC/TAC (condicionada).

TEMPORITZACIÓ

- **Sessió 9 · dimarts 13/10/2026:** presentació de la prova, preparació del projecte i parts A, B i C.
- **Sessió 10 · dimecres 14/10/2026:** parts D, E i F, i PE1 (test del RA1).
- La resta del treball es fa de manera autònoma.

Data de lliurament: dilluns 19/10/2026, a les 23:59, al Classroom.

ANNEXES

Format i lliurament

- Activitat **individual**.
- Al Classroom: el **ZIP del projecte sense la carpeta target**, amb el nom `PR1_AEA1_0486_Cognom1Cognom2_Nom.zip`, i l'**enllaç al repositori** de GitHub.
- Si el repositori és privat, afegeix el docent com a col·laborador (usuari `ilopez000`).
- El docent pot demanar una explicació oral breu de qualsevol forat. Si no saps explicar un codi que has lliurat, aquell forat no compta.
- Retorn de la nota amb la rúbrica emplenada i comentaris per criteri.

Annex 1 · Sortida esperada de GameVaultApp

Amb tots els forats fets, `GameVaultApp` mostra això. La marca de temps de la còpia serà la del moment en què l'executis; a Windows, algunes mides poden variar uns bytes pels salts de línia i les rutes surten amb «\».

Consola

```
GameVault v1 · PR1 Accés a dades
================================
1. Estructura preparada: import, export i backup
2. Còpia de seguretat: catalog_20261008-113834.csv
3. catalog_brut.csv: 2 vàlids, 4 descartats
     línia 3: la nota ha d'anar de 0 a 10 (12.0)
     línia 4: incompleta
     línia 5: data mal formada
     línia 7: les hores no poden ser negatives (-5.0)
4. catalog.csv: 5 jocs
5. Exportat a JSON: jocs.json (864 bytes)
6. Convertit JSON → CSV: 5 jocs · iguals a l'original? sí
7. Cau binària: cataleg.ser (652 bytes) · 5 jocs recuperats
8. Índex d'hores: 5 registres de 92 bytes (460 bytes)
     posició 2 · Metroid Dread: 12.5 h + 3.0 h = 15.5 h
9. Inventari (3574 bytes en total):
     backup/catalog_20261008-113834.csv (329 bytes)
     cataleg.ser (652 bytes)
     catalog.csv (329 bytes)
     catalog_brut.csv (341 bytes)
     export/hores.dat (460 bytes)
     export/jocs.json (864 bytes)
     export/tornada.csv (329 bytes)
     operacions.log (270 bytes)
10. Error controlat: Format no suportat: jocs.xml
Registre de l'execució: dades/operacions.log
```

Annex 2 · Els forats un per un

TODO	Classe · mètode	Què has de fer	Proves	CA
PR1-01	Joc · constructor	Validar títol, hores i nota	JocTest (4)	1.6
PR1-02	Magatzem · preparaEstructura	Crear import, export i backup	MagatzemTest (1)	1.1
PR1-03	Magatzem · copiaSeguretat	Còpia amb marca de temps	MagatzemTest (1)	1.1

TODO	Classe · mètode	Què has de fer	Proves	CA
PR1-04	Magatzem · midaTotal	Sumar mides amb Files.walk	MagatzemTest (1)	1.1
PR1-05	RegistreOperacions · registra	Línia al log sense esborrar	RegistreOperacionsTest (2)	1.4
PR1-06	CsvJocDao · llegeix	Jocs vàlids i incidències	CsvJocDaoTest (2)	1.3 · 1.6
PR1-07	CsvJocDao · escriu	Capçalera i files amb CSVWriter	CsvJocDaoTest (2)	1.4
PR1-08	JsonJocDao · creaMapper, escriu	Configurar Jackson i escriure	JsonJocDaoTest (1)	1.4 · 1.5
PR1-09	JsonJocDao · llegeix	Llegir una List<Joc>	JsonJocDaoTest (2)	1.3
PR1-10	Convertidor · desa, converteix	Desar per extensió i convertir	ConvertidorTest (3)	1.5 · 1.6
PR1-11	CauCataleg · desa, carrega · Joc	Serialitzar i desserialitzar	CauCatalegTest (3)	1.3 · 1.4
PR1-12	IndexHores · llegeixHores	Llegir amb seek	IndexHoresTest (2)	1.2
PR1-13	IndexHores · sumaHores	Actualitzar només 8 bytes	IndexHoresTest (1)	1.2
PR1-14	ProvesPropiesTest	Dues proves teves	les teves (2)	1.7
PR1-15	README.md	Tres respostes i declaració d'IA	—	1.2 · 1.5 · 1.6

Annex 3 · Rúbrica de correcció

Cada criteri es valora amb un nivell de l'1 al 4 i una nota dins de l'interval del nivell. La nota de la prova és la suma de nota × pes.

Nivell	Interval	Valor de referència	Què vol dir
1 Insuficient	0 – 4,9	3,0	L'evidència no existeix o té errors que impedeixen donar el criteri per superat.
2 En procés	5 – 6,4	5,7	L'evidència existeix i és correcta en l'essencial, amb mancances que cal corregir.
3 Assolit	6,5 – 8,4	7,5	L'evidència és completa i correcta, amb defectes menors.
4 Excel·lent	8,5 – 10	9,3	Completa, correcta i amb criteri tècnic propi més enllà de l'enunciat.

Criteri	1 Insuficient	2 En procés	3 Assolit	4 Excel·lent
C1 · Gestió de fitxers i directoris CA 1.1 · **10 %** *PR1-02, 03 i 04 · MagatzemTest (3 proves)*	Els mètodes del Magatzem no estan fets o no funcionen. Fa servir java.io.File o munta les rutes concatenant text amb «/». Cap prova de MagatzemTest en verd.	Una o dues proves de MagatzemTest en verd. Falta algun mètode, o la còpia no segueix el format nom_marca.ext, o la segona crida a preparaEstructura falla.	Les 3 proves de MagatzemTest en verd. Fa servir Path, resolve i Files; Files.walk va dins d'un try-with-resources i la còpia conserva l'original.	Tot l'anterior i, a més, tracta casos que no es demanen (fitxer sense extensió, carpeta buida) i els demostra amb una prova pròpia. Codi net i sense repeticions.
C2 · Formes d'accés: accés aleatori CA 1.2 · **15 %** *PR1-12 i 13 · IndexHoresTest (3 proves) · pregunta 1 del README*	llegeixHores o sumaHores no estan fets, o reescriuen tot el fitxer. Sense resposta a la pregunta 1 o amb una resposta que no té relació amb el que s'ha fet.	Llegeix bé les hores, però sumaHores canvia altres bytes, fa créixer el fitxer o no torna a fer seek abans d'escriure. La resposta 1 és genèrica.	Les 3 proves d'IndexHoresTest en verd. El desplaçament es calcula amb les constants de la classe, sense números màgics escampats. La resposta 1 explica per què n'hi ha prou de tocar 8 bytes.	Tot l'anterior i, a més, la resposta 1 compara amb criteri l'accés seqüencial i l'aleatori (registres de mida fixa, cost de recórrer el CSV, quan no compensa) amb exemples propis de GameVault.
C3 · Recuperació d'informació CA 1.3 · **15 %** *PR1-06, 09 i 11 (carrega) · CsvJocDaoTest, JsonJocDaoTest i CauCatalegTest*	El CSV brut fa petar el programa o no es llegeix. El JSON o la cau no es poden recuperar.	Llegeix el CSV, però les incidències no porten el número de línia o el motiu, o el JSON retorna mapes en lloc d'objectes Joc. Alguna prova de lectura en vermell.	Totes les proves de lectura en verd: 2 jocs vàlids i 4 incidències amb el motiu, lectura JSON amb TypeReference, cau recuperada sencera. Codificació UTF-8 explícita.	Tot l'anterior i, a més, tolera casos que no es demanen (línia en blanc, espais, camps de més) i ho demostra amb una prova pròpia.
C4 · Emmagatzematge d'informació CA 1.4 · **15 %** *PR1-05, 07, 08 (escriu) i 11 (desa) · RegistreOperacionsTest, CsvJocDaoTest, JsonJocDaoTest, CauCatalegTest*	No escriu o el fitxer resultant no es pot rellegir. El registre es buida a cada execució.	Escriu el CSV o el JSON, però falla la capçalera, les cometes del títol amb «;» o la indentació. El registre no porta la data o no fa servir APPEND.	Totes les proves d'escriptura en verd. try-with-resources a tots els fluxos, UTF-8 explícit, CREATE i APPEND al registre i cau serialitzada correctament.	Tot l'anterior i, a més, el codi d'escriptura és clar, sense duplicacions, i les decisions (per què un ArrayList a la cau, per què writeNext amb false) estan explicades al Javadoc.

Criteri	1 Insuficient	2 En procés	3 Assolit	4 Excel·lent
C5 · Conversió entre formats CA 1.5 · **15 %** *PR1-08 (configuració), 10 · ConvertidorTest i JsonJocDaoTest · pregunta 2*	No converteix, o el Convertidor repeteix el codi de lectura i escriptura en comptes de fer servir els DAO. Sense resposta a la pregunta 2.	Converteix en un sol sentit, o les dates surten com a [2017,2,24], o el JSON no tolera camps de més. La resposta 2 tria un format sense justificar-lo.	Proves de conversió i de JSON en verd: CSV → JSON → CSV sense pèrdua. desa() reutilitza els DAO igual que carrega(). La resposta 2 justifica el format de cada cas.	Tot l'anterior i, a més, la resposta 2 compara els tres formats amb arguments tècnics (llegibilitat, interoperabilitat, velocitat, versió de la classe) i mides reals dels fitxers generats.
C6 · Previsió i gestió d'excepcions CA 1.6 · **15 %** *PR1-01, 06 (motius), 10 · JocTest i proves d'error · pregunta 3*	Hi ha catch buits, printStackTrace o catch (Exception e) genèrics. El programa peta amb dades invàlides o el Joc accepta dades incorrectes.	Controla part dels errors, però perd la causa original o dona missatges poc clars. Alguna prova de JocTest o d'error en vermell. La resposta 3 és incompleta.	JocTest i les proves d'error en verd (extensió no suportada, error d'escriptura, cau corrompuda, posició fora del fitxer). CatalegException amb causa, captures diferenciades i resposta 3 correcta.	Tot l'anterior i, a més, una estratègia coherent: cada capa llança el tipus d'excepció que li toca, els missatges diuen què ha passat i on, i la resposta 3 segueix el camí de l'error pel codi.
C7 · Proves i documentació CA 1.7 · **15 %** *PR1-14 i 15 · Javadoc · README · historial de commits*	Sense proves pròpies o en vermell. README sense respostes ni declaració d'IA. Sense Javadoc. Un sol commit amb tot el projecte o cap repositori.	Una sola prova pròpia, o proves trivials que no comproven res (assertTrue(true)). Respostes molt curtes. Javadoc a pocs mètodes. Pocs commits i concentrats al final.	Dues proves pròpies significatives en verd. Javadoc en català als mètodes completats amb @param, @return i @throws. README complet. Mínim 5 commits repartits amb missatges que diuen què canvia.	Tot l'anterior i, a més, les proves pròpies cobreixen casos límit o d'error rellevants, el Javadoc aporta informació que no és al codi i l'historial explica per si sol com s'ha fet la pràctica.

Full de correcció

Alumne/a: _____ · Repositori: _____

Criteri	Pes	Nivell (1-4)	Nota (0-10)	Nota × pes
C1 · Gestió de fitxers i directoris (CA 1.1)	10 %			
C2 · Formes d'accés: accés aleatori (CA 1.2)	15 %			
C3 · Recuperació d'informació (CA 1.3)	15 %			
C4 · Emmagatzematge d'informació (CA 1.4)	15 %			
C5 · Conversió entre formats (CA 1.5)	15 %			
C6 · Previsió i gestió d'excepcions (CA 1.6)	15 %			
C7 · Proves i documentació (CA 1.7)	15 %			
Nota de la PR1	**100 %**			**___ / 10**

Condicions de correcció

- Si el projecte **no compila**, la nota màxima de la PR1 és **3**.
- Si es modifica alguna prova donada (o `GameVaultApp`) per fer-la passar, el criteri afectat val **0**.
- Cada criteri per sota de 5 es marca com a CA no assolit i s'ha de recuperar.

- Codi que no saps explicar en la defensa oral: aquell forat no compta.
- **Recuperació:** prova pràctica equivalent, corregida amb aquesta rúbrica, als dies de recuperació de RA (30-31/03 i 1-2/04/2027) o abans si el docent ho indica.

Annex 4 · Compte amb això

- Sense `StandardCharsets.UTF_8`, els accents es trenquen en un ordinador i no en un altre.
- Sense `StandardOpenOption.APPEND` el registre es buida a cada execució; sense `CREATE`, falla el primer cop.
- `NumberFormatException` és filla d'`IllegalArgumentException`: el seu `catch` ha d'anar **abans**, o el compilador es queixarà que ja està capturada.
- Sense el `JavaTimeModule`, Jackson diu *Java 8 date/time type not supported by default*.
- Amb `List.class` en lloc de `TypeReference`, Jackson retorna una llista de `LinkedHashMap`, no de `Joc`.
- Si `Joc` no implementa `Serializable`, la cau falla amb `NotSerializableException`.
- `writeChars` escriu **2 bytes per caràcter**: el títol de 40 caràcters ocupa 80 bytes.
- Després de `readDouble` el punter ha avançat 8 bytes: torna a fer `seek` abans d'escriure.
- Fes un commit cada vegada que tanquis un forat. L'historial també s'avalua.